

[🏠 Workshop Home](#) > [Day 1](#) > [Day 2](#)

# Day 2 — Autonomous & Multi-Agent AI Systems

Move from single-turn chatbots to true AI autonomy. In the morning, build agents that reason, browse the live web, and self-heal their own Python code. In the afternoon, build collaborative multi-agent arenas and reflective creator-critic duos!

 9:00 AM – 5:00 PM

3 Sessions

5 Live Projects Built

ReAct · Self-Healing Loops · Adversarial Debate · Reflection



## Day 2 Prerequisites & Package Installation

Run this command in your terminal inside your project folder before starting:

**BASH** Terminal

```
pip install langchain langchain-ollama langchain-community langgraph duckduckgo-search ddgs wikipedia
```

# Single-Agent Systems: Tools & Autonomous Reasoning

📖 CORE CONCEPTS & MENTAL MODEL



### The ReAct Loop: Reason + Act

Traditional LLMs guess answers in one shot (often hallucinating facts or math). An

#### Agent

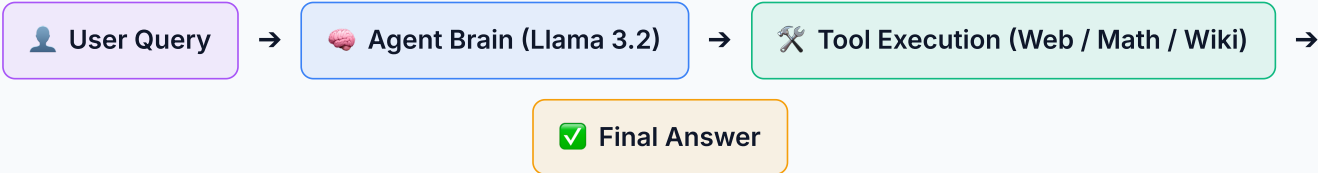
implements the

#### ReAct Loop

:

- Thought**  
: Understand the user's objective and determine what external tool is needed.
- Action**  
: Call a real Python function (search engine, Wikipedia, calculator, database).
- Observation**  
: Read the tool output and evaluate whether more information is needed.
- Final Answer**  
: Formulate a grounded, factual response.

### Single-Agent ReAct Workflow



⚡ PROJECT 1: PERSONAL RESEARCH ASSISTANT AGENT



### Personal Research Assistant Agent

An autonomous agent equipped with DuckDuckGo Search, Wikipedia, and an exact math calculator.

1

### Create day2\_01\_research\_assistant.py

This program defines custom tools with python decorators and links them to Chat01lama using LangGraph's prebuilt ReAct agent.

```
# =====
# Day 2 - Program 1: Personal Research Assistant Agent
# Concept: Single-Agent ReAct (Reason + Act) Loop with Tool Calling
# =====

# 1. Suppress internal deprecation warnings to keep student terminal output clean
import warnings
warnings.filterwarnings("ignore")

import math
from langchain_ollama import ChatOllama
from langchain_core.tools import tool
from langchain_community.tools import WikipediaQueryRun, DuckDuckGoSearchRun
from langchain_community.utilities import WikipediaAPIWrapper
from langgraph.prebuilt import create_react_agent

# — STEP 1: Define Custom Python Tools —
# The @tool decorator automatically generates JSON Schema from docstring and type hint
# The LLM reads the docstring to understand WHEN and HOW to call the tool.
@tool
def calculator(expression: str) → str:
    """Evaluates a mathematical expression safely.
    Use this for any calculations. Input: math expression as string.
    Examples: '25 * 4', 'math.sqrt(256)', '(100 + 200) / 3'"""
    try:
        # Safely restrict globals to prevent arbitrary system commands
        result = eval(expression, {"__builtins__": {}}, {"math": math})
        return f"Result: {result}"
    except Exception as e:
        return f"Error in calculation: {str(e)}"

# — STEP 2: Initialize External Search & Reference Tools —
# Limit Wikipedia characters to prevent overwhelming local model context window
wiki_wrapper = WikipediaAPIWrapper(top_k_results=2, doc_content_chars_max=1000)
wiki_tool = WikipediaQueryRun(api_wrapper=wiki_wrapper)
search_tool = DuckDuckGoSearchRun()

# Pack all tools into a list to provide to the agent
tools = [search_tool, wiki_tool, calculator]

# — STEP 3: Initialize Local LLM & ReAct Agent —
# temperature=0 ensures deterministic, hallucination-free tool selection
llm = ChatOllama(model="llama3.2", temperature=0)

# create_react_agent binds tools to the LLM and manages the Reason → Act → Observe loop
agent = create_react_agent(llm, tools)

# — STEP 4: Interactive User Loop —
def main():
    print("👋 Personal Research Assistant Agent")
    print("=" * 50)
    print("Tools available: Web Search (DuckDuckGo), Wikipedia, Math Calculator")
    print("Type 'quit' or 'exit' to stop.\n")

    while True:
        query = input("🗣️ You: ").strip()
        if query.lower() in ['quit', 'exit']:
            print("👋 Goodbye!")
            break
```

```

        if not query:
            continue

    print("\n🤖 Agent working...\n")
    # Invoke agent by passing user prompt in standard messages format
    result = agent.invoke({"messages": [("user", query)]})

    # The final response is the content of the last message in the agent's state
    print(f"\n✅ Final Answer:\n{result['messages'][-1].content}")
    print("\n" + "=" * 50 + "\n")

if __name__ == "__main__":
    main()

```

2

## Run & Test

BASH

```
python code_samples/day2_01_research_assistant.py
```

### ✅ VERIFIED SAMPLE OUTPUT

OUTPUT Terminal Output

```

🤖 Personal Research Assistant Agent
=====
Tools available: Web Search (DuckDuckGo), Wikipedia, Math Calculator
Type 'quit' or 'exit' to stop.

👤 You: What is the square root of 256 multiplied by 10?
🤖 Agent working...

✅ Final Answer:
The square root of 256 multiplied by 10 is 160.

```

## ⚡ PROJECT 2: THE AI FACT-CHECKER & WEB DETECTIVE



### The AI Fact-Checker & Web Detective

Paste any viral internet claim, rumor, or trivia. The agent browses the web, gathers evidence, and delivers a structured verdict.

1

### Create day2\_02\_fact\_checker.py

Uses resilient search handlers and custom system prompts to force structured, objective journalistic verification reports.

```

# =====
# Day 2 - Program 2: The AI Fact-Checker & Web Detective
# Concept: Domain-Specific Agent with Enforced Reporting Structure
# =====

# 1. Suppress internal deprecation notices
import warnings
warnings.filterwarnings("ignore")

from langchain_ollama import ChatOllama
from langchain_core.tools import tool
from langchain_community.tools import DuckDuckGoSearchRun
from langgraph.prebuilt import create_react_agent
import wikipedia

# — STEP 1: Resilient Search & Encyclopedia Tools —
# Pro-Tip for Local LLMs: We use parameter name `topic` instead of `query`
# because small 3B models can confuse the word 'query' with internal schema tokens.
@tool
def search_web(topic: str) → str:
    """Searches the live web for breaking news, claims, and articles."""
    # Defensive type check: unwrap dictionary if local model passed JSON object
    if isinstance(topic, dict):
        topic = str(list(topic.values())[0])
    try:
        search = DuckDuckGoSearchRun()
        return search.run(str(topic))
    except Exception as e:
        return f"Web search notice: {e}"

@tool
def lookup_wikipedia(topic: str) → str:
    """Looks up encyclopedic facts, biographies, and history on Wikipedia."""
    if isinstance(topic, dict):
        topic = str(list(topic.values())[0])
    try:
        # Set custom user agent required by Wikimedia foundation policy
        wikipedia.set_user_agent("AgenticAIWorkshop/1.0 (student-workshop)")
        return wikipedia.summary(str(topic), sentences=3)
    except Exception as e:
        return f"Wikipedia notice: {e}"

tools = [search_web, lookup_wikipedia]

# — STEP 2: Detective Persona & Structured Output Prompt —
# Enforcing structured sections ensures the agent always provides a clean, audit-ready
system_prompt = """You are an elite AI Fact-Checker and Investigative Journalist.
Your job is to verify or debunk claims using your web search and Wikipedia tools.

Always conclude with this EXACT format:
🔍 VERDICT: [TRUE / FALSE / MISLEADING / UNVERIFIED]
📊 CONFIDENCE: [HIGH / MEDIUM / LOW]
📄 FACT-CHECK SUMMARY: (2-3 concise sentences explaining the evidence)
🔗 EVIDENCE & SOURCES: (Cite specific verified facts and sources)"""

# Initialize local Llama 3.2 model
llm = ChatOllama(model="llama3.2", temperature=0)
agent = create_react_agent(llm, tools, prompt=system_prompt)

```

# — STEP 3: Verification Runner Function —

```
def verify_claim(claim: str):
    print(f"\n🔍 Investigating Claim: \"{claim}\"")
    print("⌚ Gathering evidence from live web & Wikipedia...\n")
    # Agent reasons over evidence and generates the structured report
    response = agent.invoke({"messages": [("user", f"Fact-check this claim: {claim}")]})
    verdict = response["messages"][-1].content
    print("=" * 60)
    print("📄 FACT-CHECK REPORT")
    print("=" * 60)
    print(verdict)
    print("=" * 60 + "\n")

def main():
    print("🤖 AI Fact-Checker & Web Detective")
    print("=" * 50)
    claim = input("🗨 Enter claim to verify: ").strip()
    if claim:
        verify_claim(claim)

if __name__ == "__main__":
    main()
```

2

## Run & Test with Real Claims

BASH

```
python code_samples/day2_02_fact_checker.py
```

### ✅ VERIFIED SAMPLE OUTPUT

OUTPUT Terminal Output

```
🗨 Enter claim to verify: Python was created by Guido van Rossum in 1991.
🔍 Investigating Claim: "Python was created by Guido van Rossum in 1991."
⌚ Gathering evidence from live web & Wikipedia...

=====
📄 FACT-CHECK REPORT
=====
⚖ VERDICT: TRUE
📊 CONFIDENCE: HIGH
📄 FACT-CHECK SUMMARY: Python was indeed created by Guido van Rossum, and according to
🔗 EVIDENCE & SOURCES:
- Wikipedia: Guido van Rossum
- Python.org: About Python
=====
```

# Self-Correction & Reflection: Agents That Fix Themselves

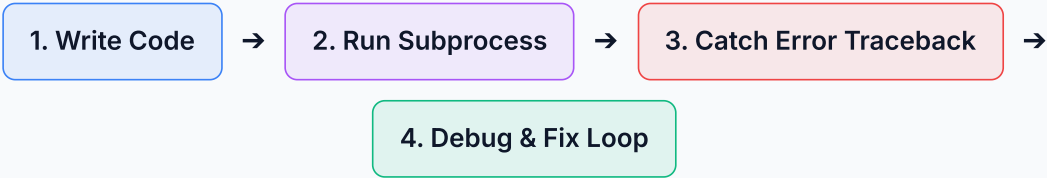
📖 CORE CONCEPTS



**The Shift: From Generation to Iterative Self-Correction**

In standard prompt engineering, if the code generates a bug or syntax error, the execution crashes. In **Agentic AI**, the agent observes its own errors in real-time, analyzes the traceback, and heals its code autonomously.

Self-Healing Execution Cycle



⚡ PROJECT 3: THE SELF-HEALING PYTHON CODER AGENT



**The Self-Healing Python Coder Agent**

Give the agent any algorithmic or data task. It writes code, tests execution in a secure subprocess, catches runtime bugs, and iterates until it achieves exit code 0.

PYTHON day2\_03\_self\_healing\_coder.py

```
# =====
# Day 2 - Program 3: The Self-Healing Python Coder Agent
# Concept: Self-Correction Loop via Subprocess Execution & Error Feedback
# =====

# 1. Suppress internal deprecation warnings
import warnings
warnings.filterwarnings("ignore")

import re, sys, subprocess
from langchain_ollama import ChatOllama
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Low temperature (0.2) ensures code precision and fewer syntax mistakes
llm = ChatOllama(model="llama3.2", temperature=0.2)

# — STEP 1: Code Block Extraction Utility —
# Uses regex to isolate raw python code from any surrounding conversational prose
def extract_python_code(llm_output: str) → str:
    match = re.search(r"```python\s*(.*?)\s*```", llm_output, re.DOTALL)
    return match.group(1).strip() if match else llm_output.strip()

# — STEP 2: Secure Local Code Runner —
# Runs the generated script in an isolated subprocess with a 10s timeout to catch infinite loops
def execute_code(code: str) → tuple[bool, str]:
    try:
        proc = subprocess.run(
            [sys.executable, "-c", code],
            capture_output=True, text=True, timeout=10
        )
        # Exit code 0 means clean execution without exceptions
        if proc.returncode == 0:
            return True, proc.stdout.strip()
        else:
            # Return stderr containing the exact Python traceback
            return False, proc.stderr.strip()
    except Exception as e:
        return False, str(e)

# — STEP 3: Self-Healing Agentic Loop —
def self_healing_coder(user_task: str, max_attempts: int = 3):
    print("=" * 60)
    print(f"🔴 TASK: {user_task}")
    print("=" * 60)

    # Chain 1: Initial Code Generator
    coder_prompt = ChatPromptTemplate.from_messages([
        ("system", "You are an expert Python Engineer. Write clean, executable code wr",
        ("human", "Task: {task}")
    ])

    # Chain 2: Bug Fixer & Error Healer (receives broken code + compiler error)
    fixer_prompt = ChatPromptTemplate.from_messages([
        ("system", "You are a senior debugging specialist. The previous code produced",
        ("human", "Previous Code:\n```python\n{code}\n```\n\nExecution Error Traceback:"
    ])

```

```

coder_chain = coder_prompt | llm | StrOutputParser()
fixer_chain = fixer_prompt | llm | StrOutputParser()

current_code = ""
last_error = ""

# Iterative reflection cycle
for attempt in range(1, max_attempts + 1):
    print(f"\n🔄 [Attempt {attempt}/{max_attempts}] Generating code...")
    if attempt == 1:
        raw = coder_chain.invoke({"task": user_task})
    else:
        # Pass the broken code and error traceback back into the model to self-correct
        raw = fixer_chain.invoke({"code": current_code, "error": last_error})

    current_code = extract_python_code(raw)
    print("🖨️ Executing code in subprocess...")
    success, output = execute_code(current_code)

    if success:
        print(f"\n🎉 CODE EXECUTED SUCCESSFULLY ON ATTEMPT {attempt}")
        print("=" * 60)
        print(current_code)
        print("-" * 60)
        print(f"📄 Output:\n{output}")
        print("=" * 60)
        return True
    else:
        # Capture error for the next healing attempt
        print(f"❌ Execution Failed on attempt {attempt}! Error:\n{output}")
        last_error = output

return False

if __name__ == "__main__":
    task = "Calculate the sum of squares of even numbers between 1 and 10, format as a list"
    self_healing_coder(task)

```

2

## Run & Watch the Self-Healing Loop

BASH

```
python code_samples/day2_03_self_healing_coder.py
```

✅ VERIFIED SAMPLE OUTPUT

```
=====
🔧 TASK: Calculate the sum of squares of even numbers between 1 and 10, format as a JSON
=====

🔄 [Attempt 1/3] Generating code...
🖥️ Executing code in subprocess...

🎉 CODE EXECUTED SUCCESSFULLY ON ATTEMPT 1
=====
import json

def calculate_sum_of_even_squares():
    total = 0
    for i in range(2, 11, 2):
        total += i ** 2
    result = {'sum_of_even_squares': total}
    print(json.dumps(result))

calculate_sum_of_even_squares()
-----
📄 Output:
{"sum_of_even_squares": 220}
=====
```

SESSION 3 OF 3

🕒 2:00 PM – 5:00 PM

# Multi-Agent Systems: Collaboration, Debate & Reflection

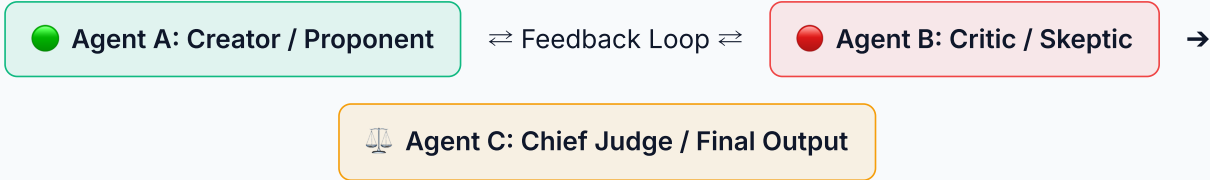
## 📖 WHY MULTI-AGENT SYSTEMS OUTPERFORM SINGLE PROMPTS



### The Society of Mind (Multi-Agent Patterns)

When a single LLM tries to write, critique, and judge its own work, it suffers from self-confirmation bias. By breaking down the task into specialized agents with distinct personas and opposing goals:

- **Adversarial Collaboration**  
: Debater agents expose blind spots that a single model misses.
- **Creator-Critic Reflection**  
: The writer focuses purely on creativity, while the critic ruthlessly enforces quality rubrics.



⚡ PROJECT 4: THE AI DEBATE ARENA ("BULL VS. BEAR + THE CHIEF JUDGE")



The AI Debate Arena

Two AI agents engage in direct verbal combat over controversial tech topics, while an impartial Chief Judge evaluates logic, delivers a scorecard, and crowns a winner.

PYTHON day2\_04\_debate\_arena.py

```

# =====
# Day 2 - Program 4: The AI Debate Arena ("Bull vs. Bear + The Chief Judge")
# Concept: Multi-Agent Adversarial Debate & Impartial Evaluation
# =====

import sys
from langchain_ollama import ChatOllama
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Higher temperature (0.7) encourages creative rhetoric and diverse debate angles
llm = ChatOllama(model="llama3.2", temperature=0.7)

# — STEP 1: Define Opposing Agent Personas —

# Agent 1: The Bull (Technological Optimist & Innovation Advocate)
bull_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are Debater A: 'The Bull' – a passionate, visionary tech optimist."),
    ("human", "{instruction}")
])

# Agent 2: The Bear (Skeptical Industry Realist & Devil's Advocate)
bear_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are Debater B: 'The Bear' – a sharp, skeptical industry critic. Argue the opposite of the Bull's perspective."),
    ("human", "{instruction}")
])

# Agent 3: The Chief Judge (Impartial Arbitrator with Strict Rubric)
judge_prompt = ChatPromptTemplate.from_messages([
    ("system", """You are 'The Chief Debate Judge'. Evaluate strictly on:
1. Logical consistency
2. Quality of evidence/examples
3. Counter-rebuttal strength
Provide a scorecard with scores out of 10 for both sides, then declare an official WINNER based on the highest total score. Be impartial and concise.

Topic: {topic}
Debate Transcript: {transcript}
Deliver your verdict and scorecard now."""),
    ("human", "Topic: \"{topic}\"\\n\\nDebate Transcript:\\n{transcript}\\n\\nDeliver Judge's Verdict and Scorecard")
])

# Compose LCEL chains for all three agents
bull_chain = bull_prompt | llm | StrOutputParser()
bear_chain = bear_prompt | llm | StrOutputParser()
judge_chain = judge_prompt | llm | StrOutputParser()

# — STEP 2: Multi-Agent Message-Passing Orchestration —
def run_debate_arena(topic: str):
    print("=" * 65)
    print(f"🐮 WELCOME TO THE AI DEBATE ARENA 🐻 TOPIC: \"{topic}\"")
    print("=" * 65)

    transcript = []

    # Round 1: Opening Statements
    print("\\n[ROUND 1: OPENING STATEMENTS]")
    a_opening = bull_chain.invoke({"instruction": f"Give opening statement supporting your perspective on: {topic}."})
    print(f"🟢 Debater A (The Bull):\\n{a_opening}\\n")
    transcript.append(f"Debater A Opening:\\n{a_opening}\\n")

    # Debater B reacts directly to Debater A's opening
    b_opening = bear_chain.invoke({"instruction": f"Give opening statement opposing the Bull's perspective on: {topic}."})
    print(f"🟠 Debater B (The Bear):\\n{b_opening}\\n")
    transcript.append(f"Debater B Opening:\\n{b_opening}\\n")

    # Chief Judge evaluates
    judge_input = f"Topic: {topic}\\n\\nDebate Transcript:\\n{transcript}\\n\\nDeliver your verdict and scorecard now."
    judge_output = judge_chain.invoke(judge_input)
    print(judge_output)

    # Final Summary
    print("\\n===== DEBATE CONCLUDED =====")
    print(f"Final Verdict: {judge_output}")
    print(f"Scorecard:\\n{judge_output}")
    print(f"Winner: {judge_output}")
    print("\\n=====")

```

```

b_opening = bear_chain.invoke({"instruction": f"Give opening statement opposing '{topic}'"})
print(f"🐻 Debater B (The Bear):\n{b_opening}\n")
transcript.append(f"Debater B Opening:\n{b_opening}\n")

# Round 2: Cross-Examination & Direct Rebuttal
print("\n[ROUND 2: REBUTTALS]")
a_rebuttal = bull_chain.invoke({"instruction": f"Rebut Debater B's critique:\n'{b_opening}'"})
print(f"🐮 Debater A Rebuttal:\n{a_rebuttal}\n")
transcript.append(f"Debater A Rebuttal:\n{a_rebuttal}\n")

b_rebuttal = bear_chain.invoke({"instruction": f"Deliver final counter-strike against Debater A's rebuttal:\n'{a_rebuttal}'"})
print(f"🐻 Debater B Closing Rebuttal:\n{b_rebuttal}\n")
transcript.append(f"Debater B Rebuttal:\n{b_rebuttal}\n")

# Round 3: Chief Judge Evaluates the entire transcript
print("=" * 65 + "\n⚖️ THE CHIEF JUDGE'S SCORECARD & VERDICT\n" + "=" * 65)
verdict = judge_chain.invoke({"topic": topic, "transcript": "\n".join(transcript)})
print(verdict)

if __name__ == "__main__":
    # Accept CLI argument or fallback to default controversial topic
    topic = sys.argv[1] if len(sys.argv) > 1 else "Should colleges ban AI tools like ChatGPT?"
    run_debate_arena(topic)

```

2

## Run Debate in Terminal

BASH

```
python code_samples/day2_04_debate_arena.py "Should colleges ban AI tools?"
```

### ✓ LIVE DEBATE OUTPUT

OUTPUT Terminal Output

```

[ROUND 1: OPENING STATEMENTS]
🐮 Debater A (The Bull):
"Colleges must take proactive steps to regulate AI tools, not ban them outright - for :

🐻 Debater B (The Bear):
"Allow me to cut through the rosy rhetoric: banning AI tools is not only impractical bu

=====
⚖️ THE CHIEF JUDGE'S SCORECARD & VERDICT
=====
Official Judgement: Debater B (The Bear) WINS THIS DEBATE.
Scorecard:
- Debater A (The Bull): 7.4/10 (Inspiring vision, but lacked concrete safety mechanisms)
- Debater B (The Bear): 8.6/10 (Realistic, pragmatic critique grounded in existing clas

```



## The Writer & Strict Editor Feedback Loop

An autonomous creative duo: Creator writes drafts, while a ruthless Critic evaluates against a 10-point rubric, looping revisions until the score reaches  $\geq 8/10$ .

PYTHON day2\_05\_writer\_critic\_duo.py

```

# =====
# Day 2 - Program 5: The Writer & Strict Editor Feedback Loop (Reflective Duo)
# Concept: Evaluator-Optimizer Multi-Agent Quality Convergence Loop
# =====

import re, sys
from langchain_ollama import ChatOllama
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Balanced temperature (0.5) allows creative drafting while maintaining focus
llm = ChatOllama(model="llama3.2", temperature=0.5)

# — STEP 1: Define Generator & Evaluator Personas —

# Agent 1: The Creator (Focuses purely on ideation, hook, and technical clarity)
writer_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an elite Tech Content Creator. Write high-engagement technical",
    ("human", "{prompt_text}")
])

# Agent 2: The Critic / Senior Editor (Rigorously evaluates against strict rubric)
critic_prompt = ChatPromptTemplate.from_messages([
    ("system", """"You are a ruthless Chief Content Editor. Evaluate on:
1. Hook & Engagement
2. Technical Substance
3. Readability & Formatting

Provide:
- SCORE: X/10
- 3 SPECIFIC ACTIONABLE CRITIQUES."""),
    ("human", "Topic: {topic}\n\nDraft:\n{draft}\n\nEvaluate thoroughly:")
])

writer_chain = writer_prompt | llm | StrOutputParser()
critic_chain = critic_prompt | llm | StrOutputParser()

# — STEP 2: Score Extraction Utility —
# Uses regex to extract numerical score out of 10 from critic's evaluation
def extract_score(review: str) → float:
    match = re.search(r"SCORE:\s*(\d+(?:\.\d+)?)\s*/\s*10", review, re.I)
    return float(match.group(1)) if match else 7.0

# — STEP 3: Multi-Agent Reflection & Quality Loop —
def run_reflective_duo(topic: str, target_score: float = 8.0, max_rounds: int = 3):
    print("=" * 65)
    print(f"👤 MULTI-AGENT REFLECTION: WRITER & STRICT EDITOR DUO\n🔥 TOPIC: \"{topic}\"")
    print("=" * 65)

    current_draft = ""
    last_feedback = ""

    # Loop until score target is met or max revision rounds reached
    for round_num in range(1, max_rounds + 1):
        print(f"\n📝 [ROUND {round_num}/{max_rounds}] Creator Agent Writing...")
        if round_num == 1:
            prompt = f"Write an impactful technical post about: '{topic}'."

```

```

else:
    # Pass editor's specific critiques back into the writer's context
    prompt = f"Revise the draft about '{topic}' by incorporating feedback:\n{last_feedback}"

    current_draft = writer_chain.invoke({"prompt_text": prompt})
    print(f"🧑‍💻 [ROUND {round_num}/{max_rounds}] Editor Reviewing...")
    review = critic_chain.invoke({"topic": topic, "draft": current_draft})
    score = extract_score(review)
    print(f"📊 Score: {score}/10")

    # Convergence condition: approved if score meets target or rounds exhausted
    if score ≥ target_score or round_num == max_rounds:
        print(f"\n🏆 FINAL APPROVED DRAFT (Score: {score}/10 on Round {round_num})")
        with open("final_approved_post.md", "w") as f:
            f.write(current_draft)
        print("💾 Saved approved output to 'final_approved_post.md'\n")
        return current_draft

    # Save critique for next round's revision
    last_feedback = review
    print(f"🔄 Score {score} < {target_score}. Sending back for revision...")

if __name__ == "__main__":
    topic = sys.argv[1] if len(sys.argv) > 1 else "Why Agentic AI is the future of software"
    run_reflective_duo(topic)

```

2

## Run Reflective Loop

BASH

```
python code_samples/day2_05_writer_critic_duo.py "Why Agentic AI is the future of software"
```

### ✅ ITERATIVE QUALITY PROGRESSION

OUTPUT Terminal Output

```

👉 [ROUND 1/3] Creator Agent Writing...
🧑‍💻 [ROUND 1/3] Editor Reviewing...
📊 Score: 6.0/10
🔄 Score 6.0 < 8.0. Sending back for revision...

👉 [ROUND 2/3] Creator Agent Writing...
🧑‍💻 [ROUND 2/3] Editor Reviewing...
📊 Score: 8.0/10

🏆 FINAL APPROVED DRAFT (Achieved 8.0/10 on Round 2)
💾 Saved approved output to 'final_approved_post.md'

```

 [Agentic AI Workshop](#) · [Day 2 of 3](#) · [← Day 1](#) · [Day 3 →](#) · [Home](#)



Conducted by [Basavesh D.](#) · [Dept. of CSE](#) · [Jyothy Institute of Technology, Bengaluru](#)